

REUNIÃO INTERMÉDIA · 14 ABRIL 2026

AI Based Compilation for Custom Hardware

Deniz Günes

Projeto Integrador 25/26, L.EIC, FEUP

CGRA — Coarse-Grained Reconfigurable Array

- Grade de PEs interligados e reconfiguráveis
- Programação via **mapeamento de DFGs**
- Métrica chave: **Initiation Interval (II)**

Objetivo: minimizar II — equivale a maximizar throughput do pipeline

Por que é difícil?

- NP-completo em geral
- Search Space exponencial com o n° de nós do DFG, e arquitetura do CGRA
- Heurísticas clássicas (SA, ILP) são lentas e às vezes subótimas

Leitura de papers fundamentais:

1. **SAT-MapIt** [1] — mapeamento exato via SAT solving
2. **LISA** [2] — GNN para gerar labels de mapeamento que reduzem o search space de uma implementação de Simulated Annealing
3. **GEML** [3] — GNN para transformar o problema de mapeamento num problema de isomorfismo de grafos, usado depois numa Deep-Q Network (DQN) para mapeament com um greedy
4. **NaviMap** [4] - Itera sobre a ideia do GEML, usando branch-and-bound guiado por DQN

1. Treinar GNN do LISA para gerar labels

Pipeline

DFG's sintéticos → `synthetic_dfg_converter.py` -> **SatMap Input format** →
`gnn_label_generator.py` → **Mappings válidos** -> `run_training.sh`

4 Labels geradas pela GNN do LISA:

1. **Schedule Order** — timeslot a que um nó é mapeado
2. **Same-Level Association** — associação de nós no mesmo nível
3. **Spatial Mapping Distance** — distância espacial esperada entre PEs
4. **Temporal Mapping Distance** — distância temporal entre timesteps

2. Mapping com search space reduzido pelas labels do LISA

- Em primeiro lugar convertemos os ficheiros de input do SatMap para o formato de input da GNN do LISA.
- Corremos o gerador de features, para anotar o grafo de DFG com as features necessárias para a GNN.
- Corremos o gerador de labels, que usa a GNN treinada para gerar os 4 labels para cada nó do DFG, (apenas usamos as ultimas 2)
- Finalmente, usamos os labels para dar prune a literais do SAT-MapIt, reduzindo o search space e acelerando o solver.

Para reduzir o *search space* do SAT-MapIt, utilizo as labels de **Spatial Mapping Distance** e **Temporal Mapping Distance**, que fornecem uma estimativa da distância espacial e temporal entre pares de nós do DFG. Antes de chamar a função `solve`, é executada uma fase de *pruning* sobre os literais candidatos, eliminando combinações de mapeamentos que violam essas previsões.

Em concreto, para cada **forward edge** (`src`, `dst`) do DFG:

- obtêm-se as previsões `pred_spatial` e `pred_temporal` produzidas pela GNN;
- enumeram-se todos os pares possíveis de literais entre o nó de origem e o nó de destino;
- para cada par, calcula-se a distância espacial real através da distância de Manhattan entre PEs:

$$d_{\text{spatial}} = |\text{row}_s - \text{row}_d| + |\text{col}_s - \text{col}_d|$$

- calcula-se também a distância temporal real:

$$d_{\text{temporal}} = \text{CycleDistance}(\text{cycle}_s, \text{cycle}_d, \mathbb{I})$$

Se uma destas distâncias exceder a previsão da GNN mais a respetiva tolerância, então o par é removido do espaço de procura:

$$d_{\text{spatial}} > \text{pred_spatial} + \text{spatial_tolerance} \vee d_{\text{temporal}} > \text{pred_temporal} + \text{temporal_tolerance}$$

Quando isso acontece, é adicionada ao SAT uma cláusula de exclusão que impede esse mapeamento conjunto:

$$\neg(\text{lit}_s \wedge \text{lit}_d)$$

equivalente em CNF a:

$$\neg\text{lit}_s \vee \neg\text{lit}_d$$

1. **Avaliar o impacto das labels no tempo de mapeamento** — avaliar o número de literais pruned.
2. **Explorar como usar as 2 primeiras labels**
3. **Melhorar a definição da arquitetura do CGRA** — definir um formato para descrever arquiteturas de CGRA, e dar parse dessas na classe CGRA
4. **Testar em CGRA 8x8** — de momento a redução de cerca de 48.4% de literais é quase puramente geométrica. Como a GNN foi treinada em apenas 1000 labels, o spacial e temporal distance variam pouco, raramente sendo diferente de 1 ou 2. Num CGRA 4x4 há 256 pares ordenados 48% deles não tem manhattan distance ≤ 2 (dependendo da topologia)

REFERENCES

- [1] C. Tirelli, L. Ferretti, and L. Pozzi, “SAT-MapIt: An Open Source Modulo Scheduling Mapper for Coarse Grain Reconfigurable Architectures,” in *Proceedings of the 20th ACM International Conference on Computing Frontiers*, in CF '23. Bologna, Italy: Association for Computing Machinery, 2023, pp. 383–384. doi: [10.1145/3587135.3591433](https://doi.org/10.1145/3587135.3591433).
- [2] Z. Li, D. Wu, D. Wijerathne, and T. Mitra, “LISA: Graph Neural Network based Portable Mapping on Spatial Accelerators ,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.
- [3] M. Kou, J. Zeng, B. Han, F. Xu, J. Gu, and H. Yao, “GEMML: GNN-Based Efficient Mapping Method for Large Loop Applications on CGRA,” *2022 59th ACM/IEEE Design Automation Conference (DAC)*, pp. 337–342, 2022, [Online]. Available: <https://api.semanticscholar.org/CorpusID:251744222>

- [4] M. Kou, J. Zeng, X. Peng, W. Ji, and H. Yao, “NaviMap: Partial Order-Guided Neural Architecture via Deep Q-Networks for Efficient CGRA Mapping,” in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025, pp. 364–371. doi: [10.1109/ICCD65941.2025.00059](https://doi.org/10.1109/ICCD65941.2025.00059).